

Recursion — Group Assignment

Group members:

Background

```
class Node {
    int val;
    Node next;

    // Recursive helper – prints from this node to the end
    private void printHelper(Node node) {
        if (node == null) { return; }           // base case
        System.out.println(node.val + " ");
        printHelper(node.next);                 // recursive case
    }

    public void print() { printHelper(this); }
}
```

Tasks

Task 1 — Reverse Print

Write a new recursive method that prints the **last** node's value first and the **first** node's value last. Do not use any loops or extra data structures.

Starter signature:

```
private void reversePrintHelper(Node node) { ... }
```

```
private void reversePrintHelper(Node node) {

    if(node == null) return;

    reversePrintHelper(node.next);
    System.out.println(node.val+ " ");

}
```

Task 2 — Iterative Print

Rewrite **printHelper** as an iterative method using a loop instead of recursion. The output should be identical to the original.

Starter signature:

```
public void printIterative() { ... }
```

Hint: Use a temporary Node variable to walk the list. When does your loop stop?

```
public void printIterative(){  
  
    Node temp = this;  
  
    while(temp != null){  
        System.out.print(temp.val + " ");  
        temp = temp.next;  
    }  
  
}
```

Task 3 — Remove by Index

Write a new recursive method that removes the node at a given index (0-based) and returns its **int val**. You may assume the index is valid.

Starter signatures:

```
private int RemoveHelper(Node curNode, Node prevNode, int index, int curIndex) {  
    ...  
}  
  
public int remove(int index) {  
    return RemoveHelper(this, null, index, 0);  
}
```

Hint: Use prevNode to relink the list.

```
private int RemoveHelper(Node curNode, Node prevNode, int index,  
int curIndex){  
  
    if(index == curIndex){           // found the index to remove  
        prevNode.next = curNode.next; // remove curNode from the node list.  
  
        curNode.next = null; // Not necessary. curNode will eventually be removed  
                             // as there are no references to it.  
        return curNode.val;  
    }  
  
    return removeHelper(curNode.next, curNode, index, curIndex+1);  
}
```